Laboratório Funcional · Aula 12

Daniel Morotti Vieira · RA 24191444-2 · Turma ESOFT6S-C
Gerado em 16/10/2026, 20:12:13 · tempo registrado no cronômetro: 45:14

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

agora() / sorteia(): Não dependem apenas dos parâmetros recebidos; o retorno muda a cada execução dependendo do relógio do sistema ou do gerador aleatório.
registra(msg): Apresenta efeito colateral por ter a única função de imprimir texto na tela/saída.
poe(lista, x): Apresenta efeito colateral ao alterar o objeto original recebido (lista.append).

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

1: A-1 A-2
4: 14
7: [7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

não respondido

main.py

```python
# Prática 1 - Funções puras e formas funcionais (Sebesta 15.2 e 15.3, p. 633-637)
# Cole em https://onecompiler.com/python
from functools import reduce

contador = 0

def proximo_id(prefixo):            # IMPURA: depende de um estado que muda
    global contador
    contador += 1
    return f"{prefixo}-{contador}"

def cubo(x):                        # PURA: só depende do parâmetro
    return x * x * x

print("1:", proximo_id("A"), proximo_id("A"))
print("2:", cubo(2) + cubo(2) == 2 * cubo(2))
print("3:", (lambda x: x * x * x)(2))           # a expressão lambda da p. 635

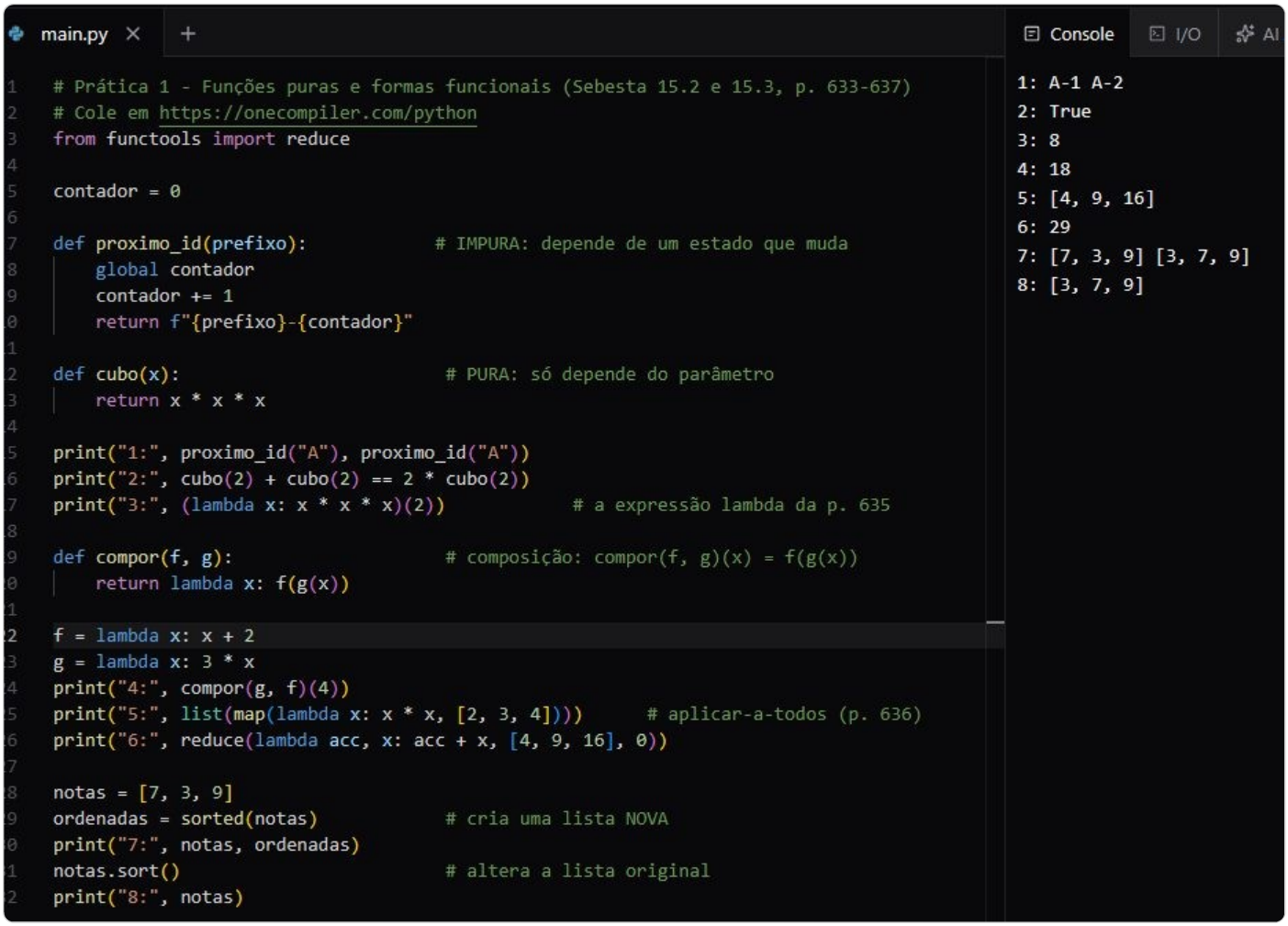
def compor(f, g):                   # composição: compor(f, g)(x) = f(g(x))
    return lambda x: f(g(x))

f = lambda x: x + 2
g = lambda x: 3 * x
print("4:", compor(f, g)(4))
print("5:", list(map(lambda x: x * x, [2, 3, 4])))      # aplicar-a-todos (p. 636)
print("6:", reduce(lambda acc, x: acc + x, [4, 9, 16], 0))

notas = [7, 3, 9]
ordenadas = sorted(notas)           # cria uma lista NOVA
print("7:", notas, ordenadas)
notas.sort()                        # altera a lista original
print("8:", notas)
```

Console I/O AI A

```
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```



1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

Altera o valor porque altera uma variável global externa (contador) a cada chamada. Dificulta testes porque o resultado depende de chamadas anteriores, quebrando o determinismo e exigindo redefinir o estado global.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f, g)(4) faz $f(g(4)) = f(12) = 14$.
compor(g, f)(4) faz $g(f(4)) = g(6) = 18$.
A ordem da aplicação altera o valor final.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria e retorna uma nova lista sem alterar a original (estilo funcional, sem efeitos colaterais).
notas.sort() modifica a lista original no próprio local de memória (estilo imperativo com efeito colateral).

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

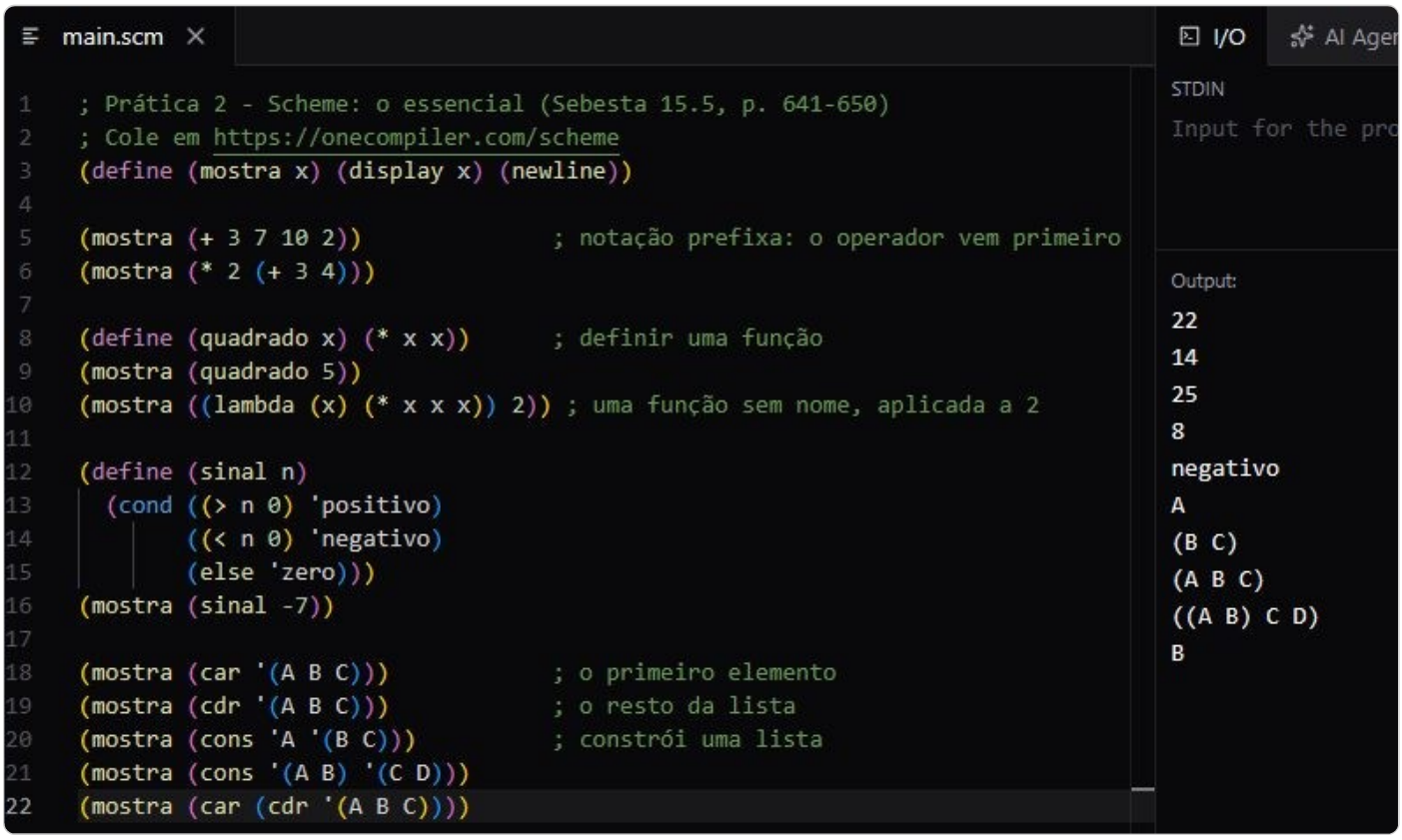
(B C)

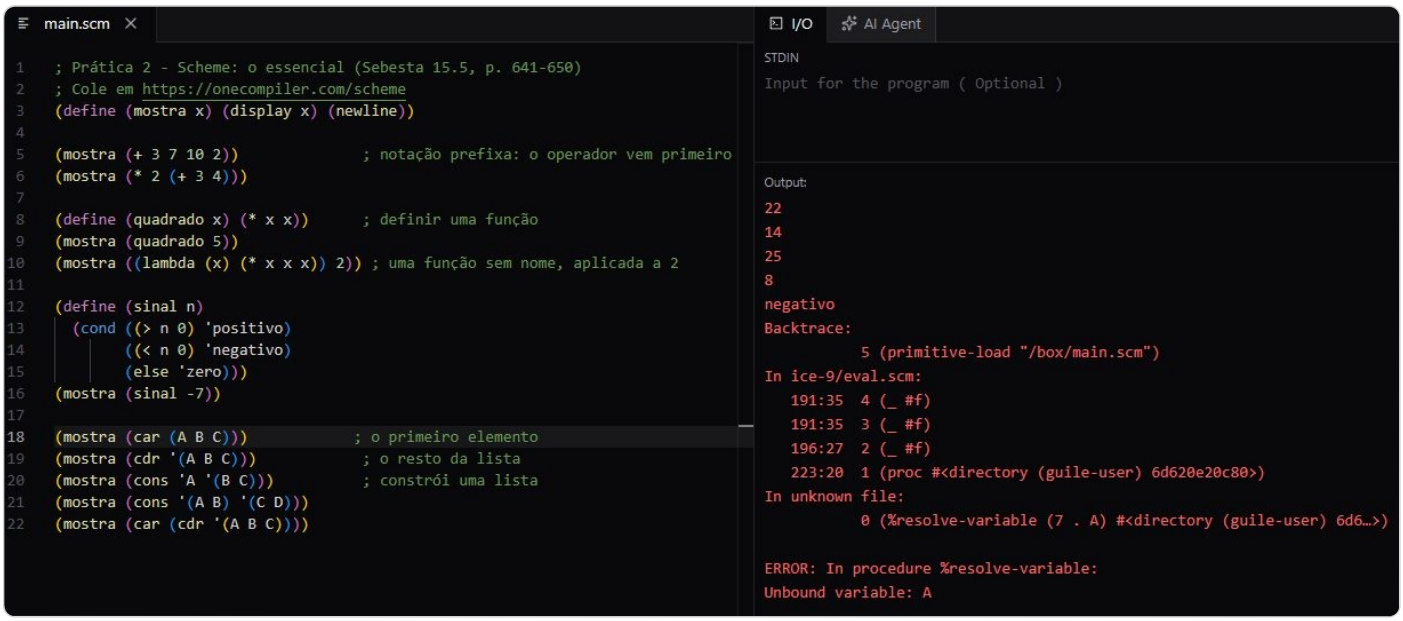
Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

não respondido





2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo indica que o termo deve ser tratado como um dado literal/lista, sem ser avaliado. Sem ele em (A B C), o Scheme tentou executar A como uma função passando B e C como parâmetros.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(mostra (car (cdr (cdr '(A B C))))) = C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

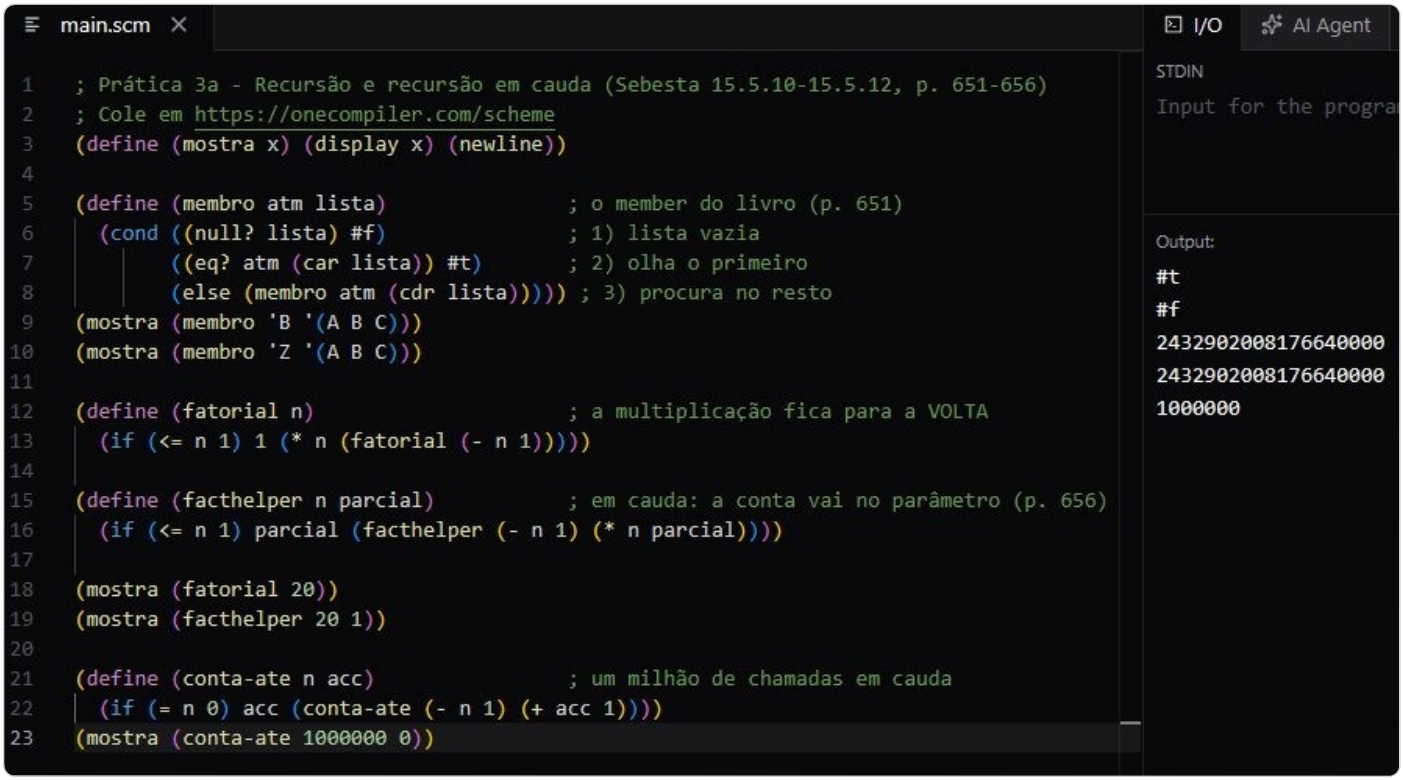
não respondido

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

No Scheme: Sim, vai funcionar. O Scheme exige a otimização de chamadas em cauda (Tail Call Optimization), reutilizando o registro de ativação na pilha e permitindo executar um milhão de chamadas sem estourar a memória. No Python: Não vai funcionar. O Python não faz a otimização de recursão em cauda e possui um limite fixo para o tamanho da pilha de chamadas, gerando o erro RecursionError.

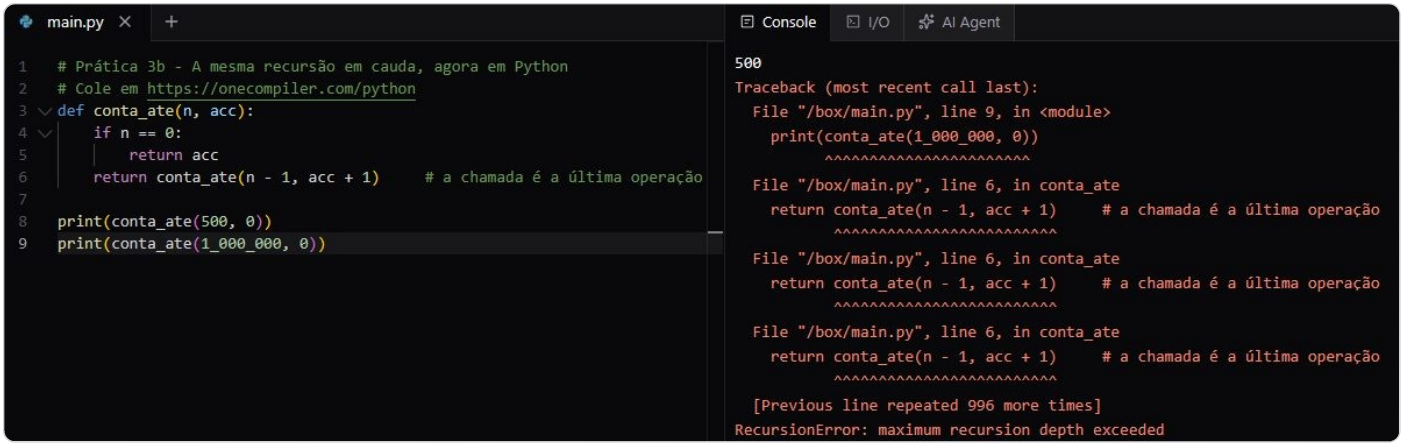
Saída do Scheme

não respondido



Saída do Python (as últimas linhas bastam)

não respondido



3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, a pilha de chamadas e a expressão com operações pendentes crescem a cada passo. Na recursão em cauda, o tamanho permanece constante porque a conta acumulada vai direto nos parâmetros.

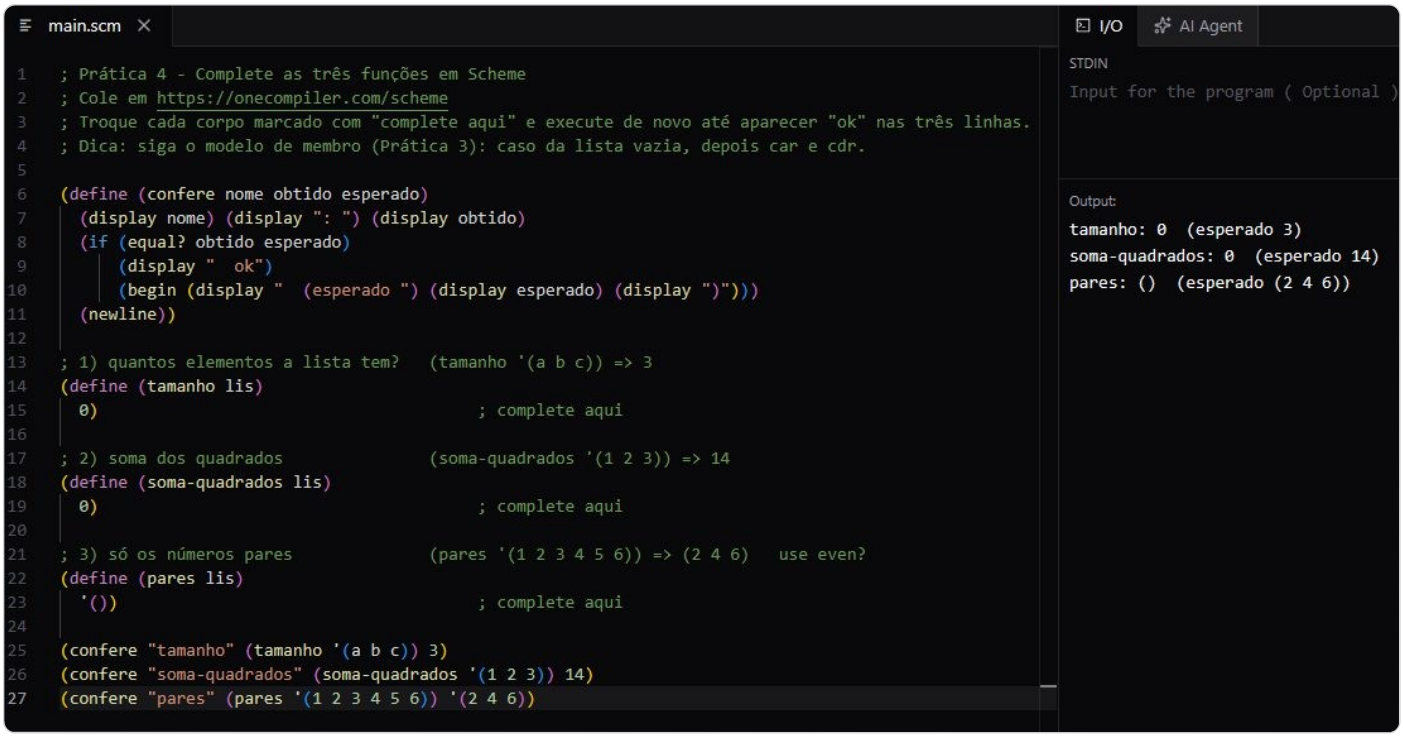
3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

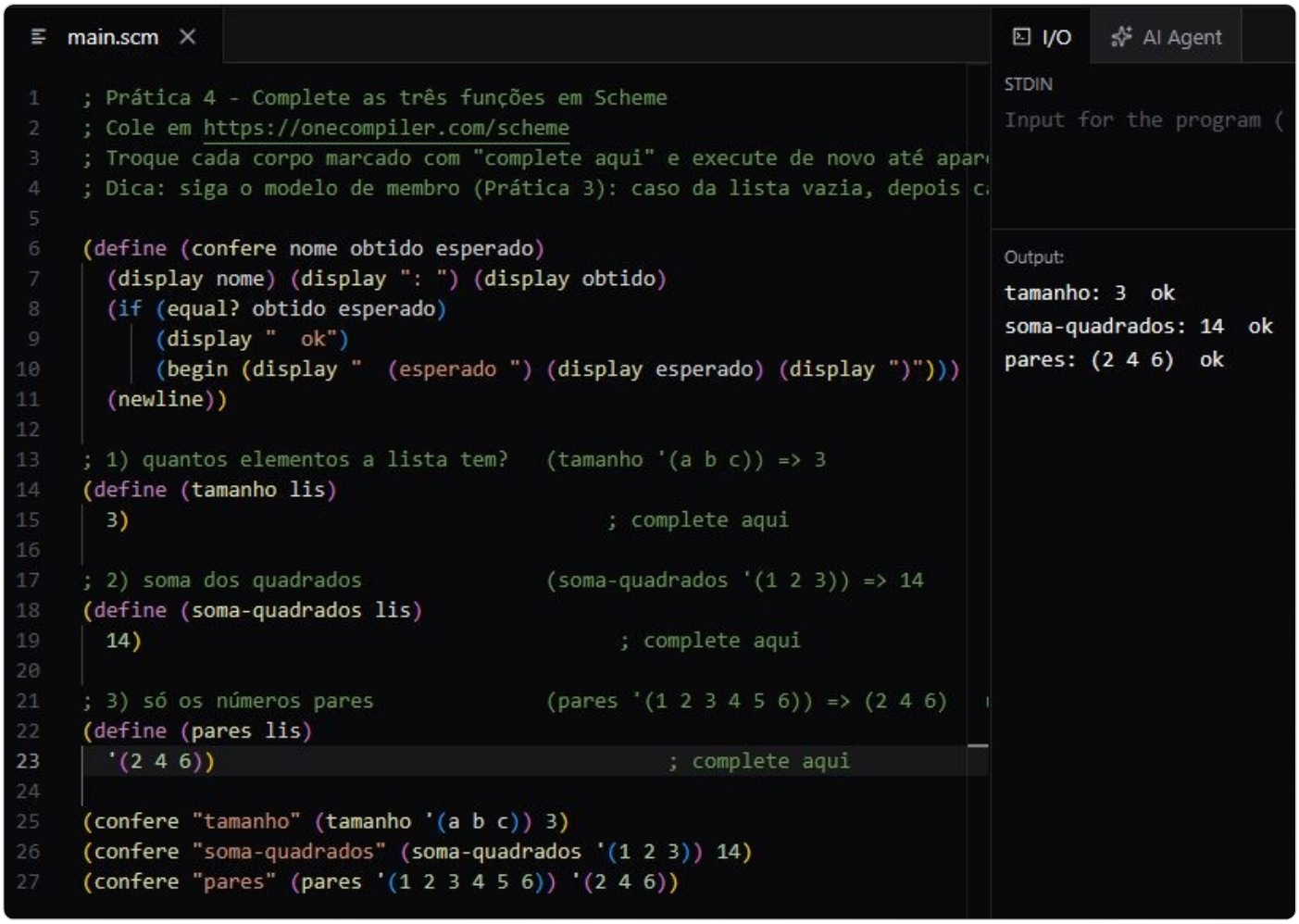
O Scheme otimiza chamadas em cauda descartando o registro de ativação anterior (transformando a chamada em iteração).
O Python não realiza essa otimização e possui um limite fixo para o tamanho da pilha de chamadas, gerando o erro

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

não respondido





4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis)) (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

Parte mais difícil: Acostumar com a sintaxe de parênteses prefixados e montar novas listas com cons sem usar variáveis acumuladoras ou laços imperativos.
Uso de null?: No início de cada função como condição de parada para checar se a lista estava vazia.
Uso de car: Para acessar o elemento atual da lista e fazer a operação (somar 1, elevar ao quadrado ou testar se é par).
Uso de cdr: Nas chamadas recursivas para passar o restante da lista.

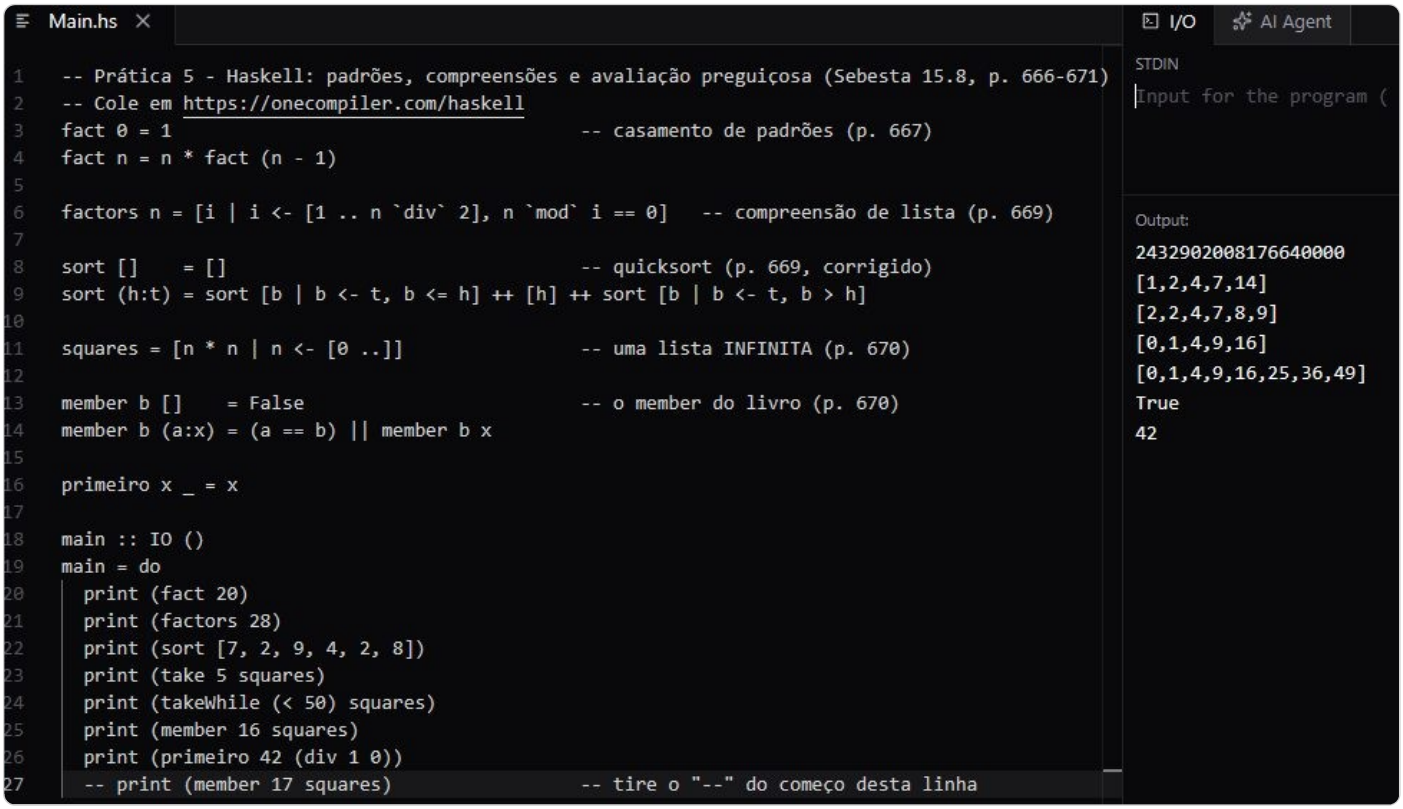
Prática 5: Haskell: padrões, compreensões e preguiça

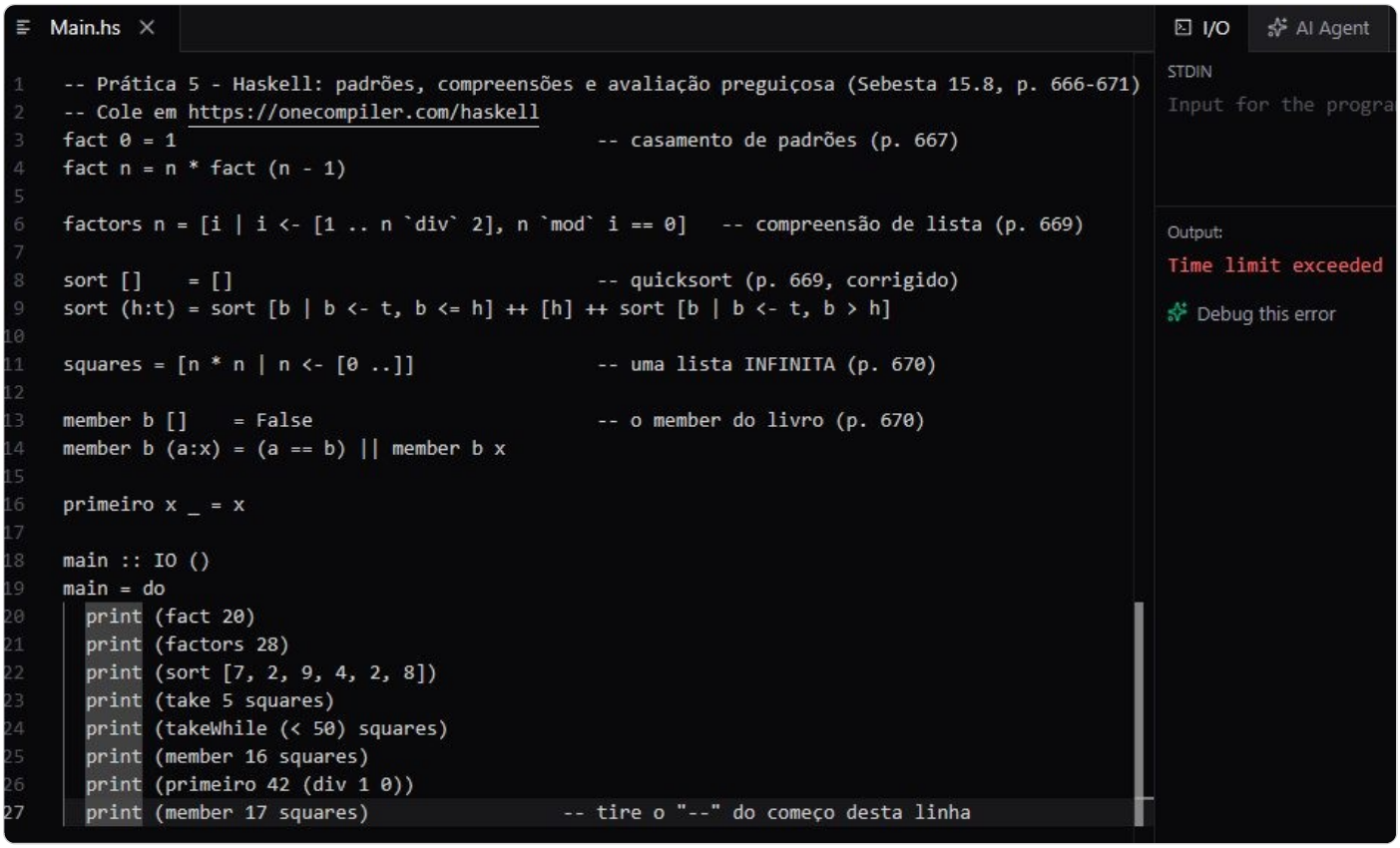
Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares imprime: [0, 1, 4, 9, 16]
primeiro 42 (div 1 0) imprime: 42
Motivo: O Haskell usa avaliação preguiçosa (lazy evaluation), então só calcula o que precisa. Por isso ele pega só os 5 primeiros quadrados da lista infinita e nem chega a avaliar a divisão por zero (div 1 0), já que a função primeiro ignora esse argumento.

Saídas (original e com member 17)

não respondido





5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell usa avaliação preguiçosa (lazy evaluation): calcula apenas os elementos estritamente necessários para o resultado.
Como a função primeiro descarta o segundo parâmetro, (div 1 0) nem chega a ser executado.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

Trava porque procura o 17 numa lista infinita e, como ele não é quadrado perfeito, a busca nunca para. O 16 parou porque foi encontrado.
Correção: Como a lista está em ordem crescente, basta interromper quando o número atual for maior que o procurado:

member b [] = False
member b (a:x)
| a == b = True
| a > b = False
| otherwise = member b x

Prática 6: Você já usa funcional

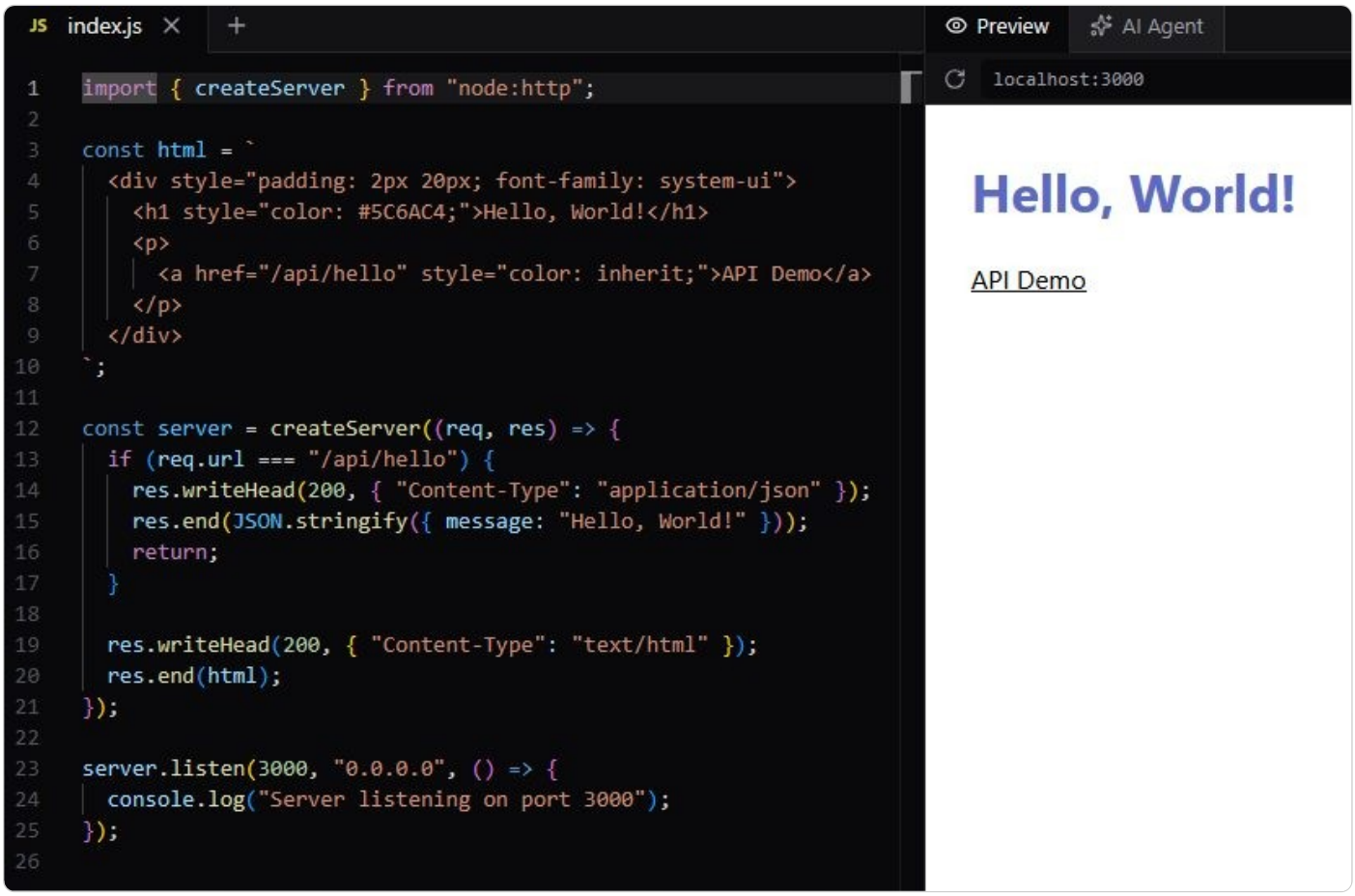
Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

ais10(5) imprime: 15
[1, 2, 3].map(vezes(5)) imprime: [5, 10, 15]
Explicação: A função somador(10) cria um closure que fixa o valor 10. Ao chamar mais10(5), soma 10 + 5. No

caso do vezes(5), o currying gera uma função que multiplica qualquer valor recebido por 5, aplicando essa transformação a cada elemento do array no map

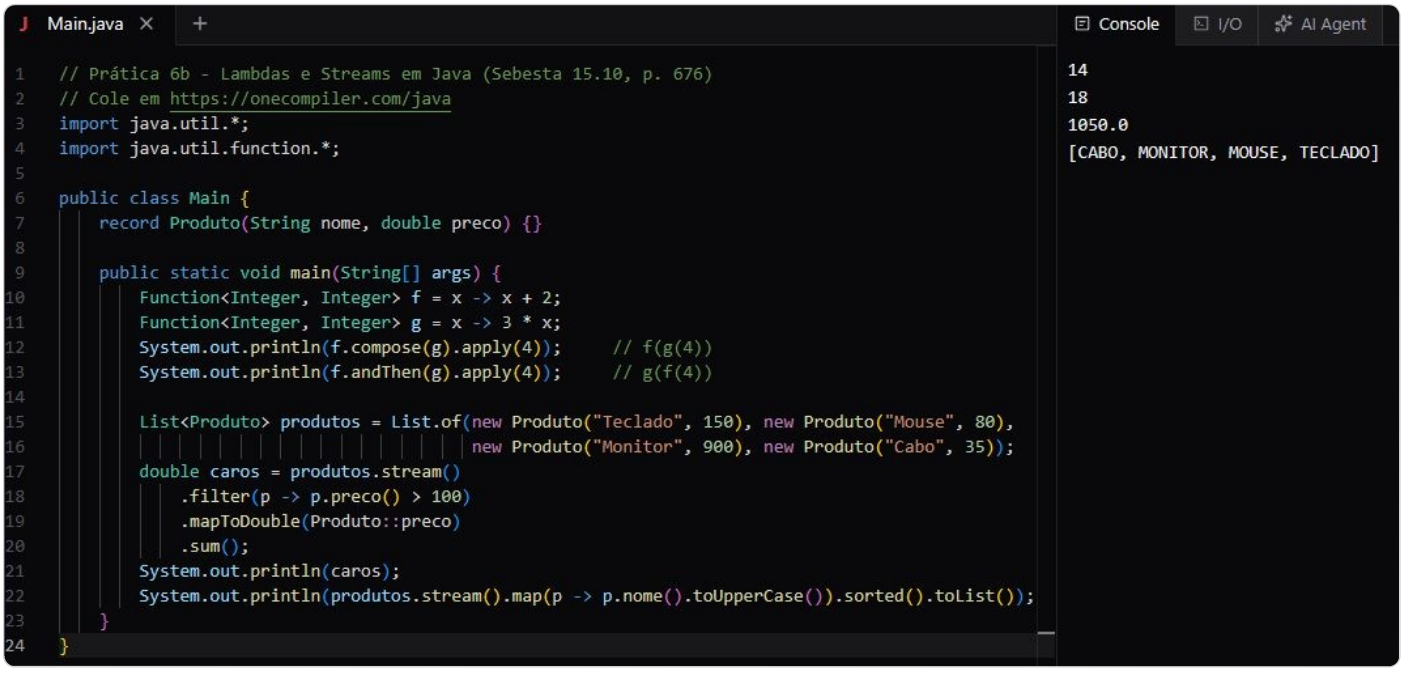
Saída do JavaScript

não respondido



Saída do Java

não respondido



6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Por causa do fechamento (closure): ao ser criada, a função interna captura e guarda o escopo onde nasceu, mantendo acesso ao valor de n.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

f.compose(g) executa g primeiro e passa o resultado para f ($f(g(x))$), exatamente como o compor(f, g) da Prática 1. f.andThen(g) executa f primeiro e passa o resultado para g ($g(f(x))$).

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();
        System.out.println(soma);
    }
}
```

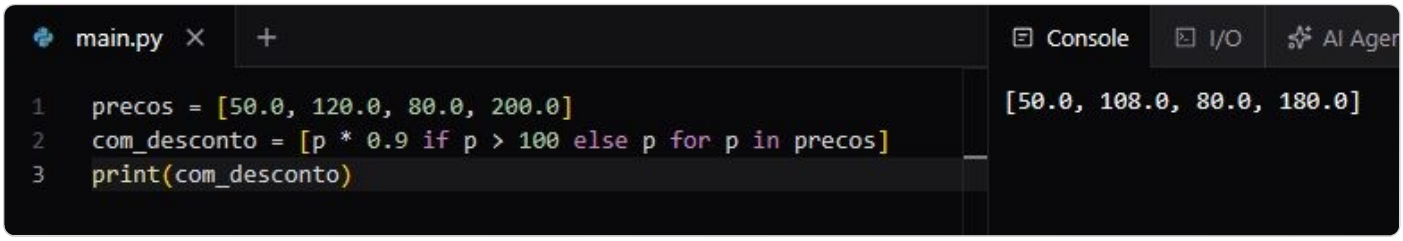
7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
precos = [50.0, 120.0, 80.0, 200.0]
com_desconto = [p * 0.9 if p > 100 else p for p in precos]
print(com_desconto)
```

Saídas das suas versões

não respondido





7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

Leitura: A versão funcional é bem mais fácil de ler, pois é declarativa e deixa claro o propósito do código (filtrar e transformar) sem precisar acompanhar índices ou contadores.

Depuração: A versão com laço costuma ser mais fácil de depurar passo a passo em depuradores tradicionais, pois permite colocar breakpoints dentro da iteração para inspecionar o valor exato das variáveis em cada ciclo.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

Legibilidade: A versão em Haskell é muito mais curta, limpa e expressiva por explicitar a intenção da conta em uma única linha.
Onde aceitar o custo (~2x mais lento): Em sistemas web, microsserviços, processamento de dados e regras de negócio onde legibilidade, manutenção simples e facilidade de paralelizar compensam a pequena diferença de desempenho.
Onde NÃO aceitar: Em motores de jogos 3D, sistemas operacionais, drivers de hardware, firmwares ou aplicações de tempo real crítico.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

A facilidade do Haskell em lidar com coleções infinitas sem consumir a memória toda por causa da avaliação preguiçosa.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

Erro: O trecho b <- h na segunda linha do código.

Correção: Trocar por b <= h.

Mensagem do compilador: O compilador acusa erro de tipo porque entende que h deveria ser uma lista de onde se extrai b (por conta do operador <-), mas h é apenas um elemento individual (o pivô).